

CI/CD

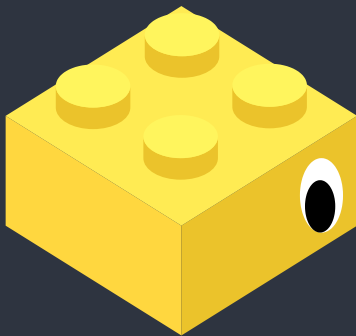
Industrialiser le cycle de vie du logiciel

Intégration continue · Livraison continue · Déploiement continu



Mathieu Campani

Software Engineer · Open Source · DuploJS



Terme important à connaître : Robustesse

Rendre **prévisible** donc **évolutif**.

Concevoir un logiciel de sorte que ses états, ses erreurs et les conséquences d'une modification puissent être déterminés à l'avance.



CI/CD

CI — Continuous Integration

Vérifier automatiquement chaque changement intégré au code.

CD — Continuous Delivery / Deployment

Préparer ou effectuer automatiquement la mise en production.



Ma première expérience avec la CI/CD

Publication de package sur NPM



1. La situation

Je publiais mes packages NPM à la main.

- Versionner.
- Builder.
- Publier.



2. Le problème

Je pouvais me tromper.

- Mauvaise version.
- Mauvaise sémantique.
- Mauvais workflow Git.



3. Première étape

Avant d'automatiser, j'ai me suis organiser.

- Branches.
- Pull requests.
- Règles de merge.



4. Encore trop manuel

Même avec GitHub, je devais encore décider moi-même :

- fix → patch
- feat → minor
- break → major



5. Première CI/CD

Une action analysait les titres de PR et calculait automatiquement la prochaine version.



6. La prise de conscience

Je suis humain, donc faillible.

La CI/CD sert à créer un processus qui reste fiable même quand moi je ne le suis pas.



Un processus qui doit être répéter

À chaque publication, je faisais la même chose.

- Une suite d'étapes.
- Des règles connues.
- Un résultat attendu.



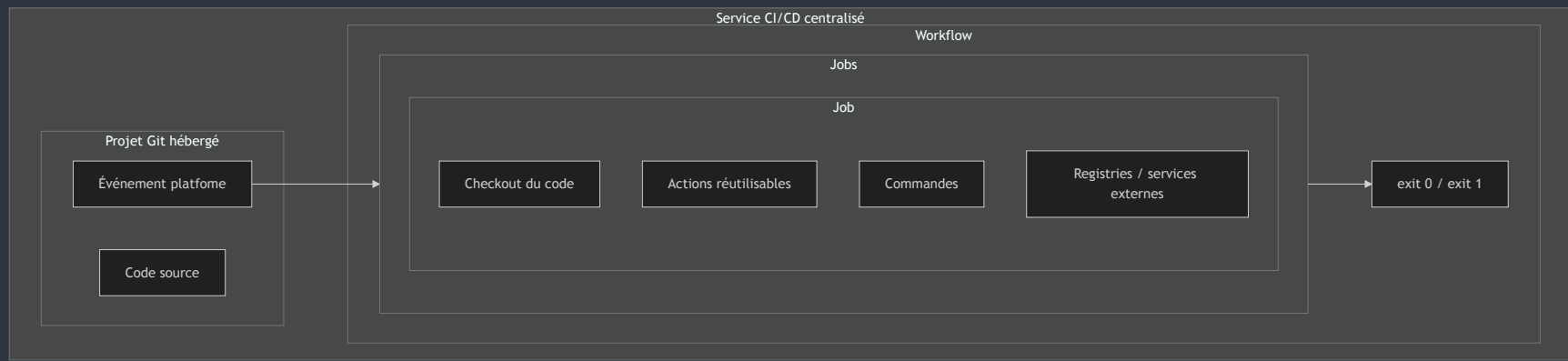
Le périmètre de la CI/CD

Une fois que le code existe, qu'est-ce qu'on en fait ?

- Vérifier.
- Construire.
- Versionner.
- Publier.
- Déployer.
- Surveiller.



Anatomie d'une CI/CD



Quels événements déclenchent une CI/CD ?

La CI/CD tourne souvent autour de Git.

Donc la question devient :

À quel moment du cycle Git veut-on déclencher un processus ?



Événements courants

Selon les plateformes, les noms changent, mais on retrouve souvent les mêmes déclencheurs :

- `push` sur une branche
- `push` d'un tag
- ouverture / mise à jour d'une pull request
- merge request
- création d'une release
- déclenchement manuel
- planification horaire
- appel externe / API / webhook



Dans un premier temps, créer
un repo de test sur GitHub.



Notre premier workflow

```
my-project/  
├─ src/  
├─ package.json  
├─ .github/  
│   └─ workflows/  
│       └─ hello-world.yml
```

```
# Meta data du workflow  
name: Hello World  
  
# Événement Déclancher  
on:  
  push:  
  
# Travaille a effectué  
jobs:  
  hello:  
    runs-on: ubuntu-latest  
    steps:  
      - run: echo "Hello World"
```



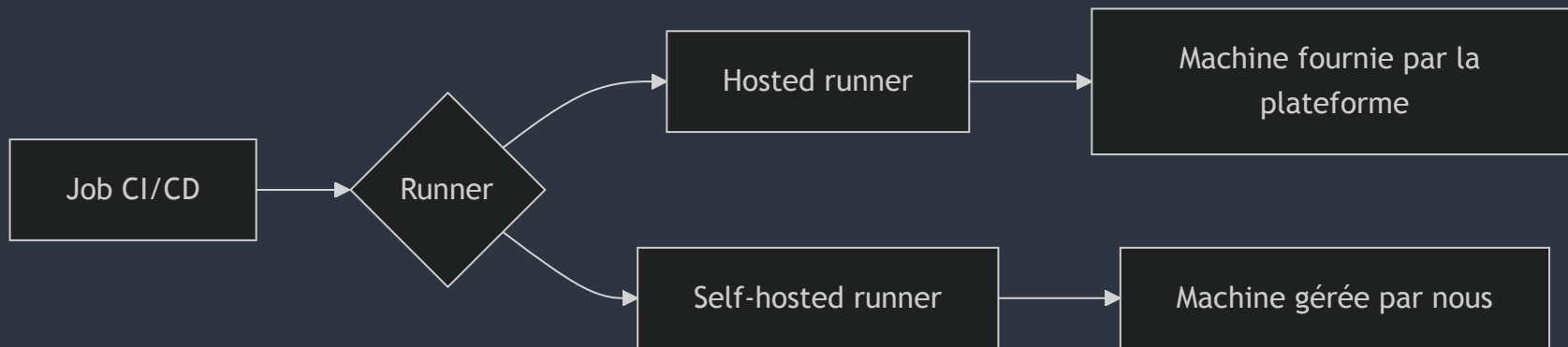
Où s'exécute un job ?

```
jobs:  
  test:  
    runs-on: ubuntu-latest
```

`runs-on` désigne la machine qui va exécuter le job.



Types de runners



GitHub / GitLab : différence importante

GitHub Actions : un runner exécute généralement **un job à la fois**.

GitLab Runner : un même runner peut exécuter **plusieurs jobs en parallèle** selon sa configuration `concurrent` .



Attention aux runners

Un runner est une machine d'exécution.

Donc il faut gérer :

- son accès au repository
- ses droits réseau
- ses secrets
- son stockage
- son nettoyage
- sa durée de vie



Où est notre code ?

Le job s'exécute sur une machine distante.

Mais par défaut, cette machine ne contient pas encore le code du repository.



Récupérer le code

```
name: Hello World
```

```
on:
```

```
  push:
```

```
jobs:
```

```
  hello:
```

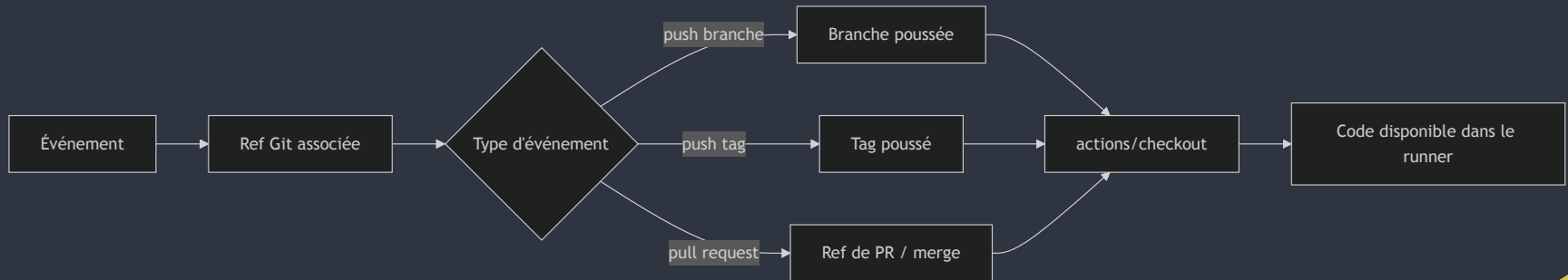
```
    runs-on: ubuntu-latest
```

```
  steps:
```

```
    # Actions qui permet de récupérer le code associé à la branche de l'événement
```

```
    - uses: actions/checkout@v7
```

```
    - run: echo "Hello World"
```



Les actions

Une action peut encapsuler presque n'importe quelle tâche réutilisable.

```
steps:
  # Récupérer le code
  - uses: actions/checkout@v7
  # Installer un environnement Node.js
  - uses: actions/setup-node@v7
    with:
      node-version: 24.18.0
  # Mettre en cache des dépendances
  - uses: actions/cache@v5
    with:
      path: ~/.npm
      key: npm-{{ hashFiles('package-lock.json') }}
  # Analyser la sécurité du code
  - uses: github/codeql-action/init@v4
  # Construire et publier une image Docker
  - uses: docker/build-push-action@v7
  # Sauvegarder un artefact
  - uses: actions/upload-artifact@v7
    with:
      name: build
      path: dist/
```



Actions spécifiques à GitHub

Certaines actions ne font pas que lancer du code.

Elles produisent des résultats directement compris par GitHub Actions.



Cache

Réutiliser des fichiers entre plusieurs exécutions.

```
- uses: actions/cache@v5
  with:
    path: ~/.npm
    key: npm-${{ hashFiles('package-lock.json') }}
```



Artefacts

Conserver le résultat d'un job.

```
- uses: actions/upload-artifact@v7
  with:
    name: build
    path: dist/
```



GitHub Pages

Déployer un site statique depuis un workflow.

```
- uses: actions/upload-pages-artifact@v4
  with:
    path: dist/

- uses: actions/deploy-pages@v5
```



SARIF / Code Scanning

Transformer un rapport d'analyse en alertes GitHub.

```
- uses: github/codeql-action/upload-sarif@v4  
  with:  
    sarif_file: results.sarif
```



Propriétés essentielles

```
steps:
  - name: Installer Node.js
    uses: actions/setup-node@v7
    with:
      node-version: 24.18.0

  - name: Lancer les tests
    run: npm test
    env:
      NODE_ENV: test

  - if: always()
    run: echo "Status : ${ job.status }"
```



Job Summary & Annotations

Afficher un résumé lisible dans GitHub Actions.

```
- run: |  
  echo "## Résultat des tests" >> $GITHUB_STEP_SUMMARY  
  echo "✅ Tous les tests sont passés" >> $GITHUB_STEP_SUMMARY
```

Faire remonter une erreur dans l'interface GitHub.

```
- run: |  
  echo "::error file=src/index.ts,line=12::Type invalide"  
  exit 1
```



Contextes GitHub

GitHub expose des valeurs utilisables dans le workflow.

```
- run: |  
  echo "Commit : ${github.sha}"  
  echo "Branche : ${github.ref_name}"  
  echo "Runner : ${runner.os}"  
  echo "Status : ${job.status}"
```

Syntaxe :

```
${contexte.propriété }
```



Passons à la pratique



Exercice 1 - Valider un projet

Créer une CI déclenchée à chaque `push`.

Le workflow doit :

- récupérer le code
- installer Node.js
- mettre en place la cache npm
- installer les dépendances
- lancer les tests
- écrire le statut du job et la ref du commit dans `$GITHUB_STEP_SUMMARY`

```
push → runner → checkout → setup → cache → install → test
```



Exercice 1 — Ce qu'il faut retenir

- `push` déclenche automatiquement le workflow à chaque nouveau commit
- chaque job s'exécute dans un runner isolé
- `checkout` rend le code du dépôt disponible dans le runner
- `setup-node` prépare la version de Node.js utilisée par le projet
- le cache npm accélère les exécutions, mais ne remplace pas `npm ci`
- une step échoue si sa commande retourne un code différent de zéro
- `if: always()` permet de produire un résumé même après un échec
- les contextes `job` et `github` donnent accès aux informations de l'exécution



Exercice 2 — Produire un artefact

Créer une CI déclenchable manuellement.

Le workflow doit :

- récupérer le code
- préparer l'environnement
- installer les dépendances
- lancer le build
- vérifiez que le fichier de build existe
- sauvegarder `dist/` comme artefact
- écrire un résumé du build (Indiqué des informations qui vous paraissent pertinentes)

```
workflow_dispatch → checkout → build → dist/ → artifact
```



Exercice 2 — Ce qu'il faut retenir

- `workflow_dispatch` permet de déclencher manuellement un workflow
- le build transforme le code source en fichiers prêts à être utilisés
- vérifier `dist/` permet d'échouer explicitement si le build n'a rien produit
- `::error::` crée une annotation visible dans l'interface GitHub
- un artefact conserve les fichiers produits après la fin du job
- `upload-artifact` sert à sauvegarder ou à partager ces fichiers
- le résumé facilite l'identification du build, du commit et de sa date



Parallélisme et partage de données

Les jobs indépendants peuvent s'exécuter en parallèle.

Mais chaque job possède son propre environnement.

```
jobs:
  test-unit:
    runs-on: ubuntu-latest
    steps:
      - run: npm run build
      - run: npm run test:unit
```

```
test-e2e:
  runs-on: ubuntu-latest
  steps:
    - run: npm run build
    - run: npm run test:e2e
```

```
# test-unit }
#           } en parallèle
# test-e2e }
```

Problème : chaque job doit refaire l'opération commune, ici **le build**.



Résolution : mutualiser le build

Un job produit le build une seule fois.

Les autres jobs le récupèrent avec un artefact.

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: npm run build
      - uses: actions/upload-artifact@v7
        with:
          name: build
          path: dist/

  test-unit:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v7
        with:
          name: build
          path: dist/
      - run: npm run test:unit
```



Stratégies d'exécution

Contrôler comment plusieurs variantes d'un job sont exécutées.

```
jobs:
  test:
    strategy:
      fail-fast: false
      max-parallel: 2

    matrix:
      node: [20, 22, 24]
      os: [ubuntu-latest, windows-latest]

    runs-on: ${ matrix.os }

    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-node@v7
        with:
          node-version: ${ matrix.node }

      - run: npm test
```



Principe du moindre privilège

GitHub crée un `GITHUB_TOKEN`
pour chaque job.

```
permissions:  
  contents: read  
  packages: write
```

Début du job

↓

`GITHUB_TOKEN` généré

↓

Permissions appliquées

↓

Exécution du job

↓

Token expiré

Un job ne doit avoir accès qu'à ce dont il a besoin.



Environments

Organiser la configuration selon la cible.

```
Repository
└─ Settings
   └─ Environments
      ├── staging
      ├── preprod
      └─ production
```

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: staging
```

Même workflow.

Même nom de variable.

Valeur différente selon l'environnement.



Variables par environnement

```
- run: echo "Deploy to ${vars.API_URL}"
```

```
staging.API_URL → https://api-staging.example.com
```

```
production.API_URL → https://api.example.com
```



Protéger une cible

Un environnement peut imposer ses propres règles.

```
production
├─ branches autorisées
├─ tags autorisés
├─ approbation requise
├─ secrets dédiés
└─ historique de déploiement
```



Une règle hors du workflow

Le YAML dit :

```
environment: production
```

Mais GitHub décide si le job peut continuer.

```
workflow → demande production
```

↓

```
règles GitHub
```

↓

```
autorisé / bloqué
```



CI ou CD ?

Les Environments peuvent aider en CI.

Mais leur intérêt principal est en CD :

contrôler ce qui a le droit d'atteindre une cible.

build → test → deploy



environment



production

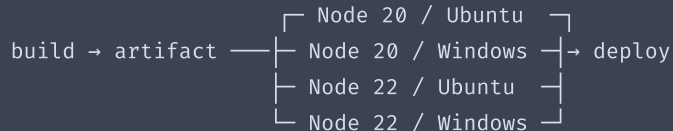


Exercice 3 - Construire, tester et déployer

Une application Node.js produit son build dans `dist/`.

Créer un workflow déclenchable manuellement avec un paramètre permettant de choisir entre les environnements `staging` et `production`.

Le pipeline doit construire l'application **une seule fois**, tester le même build sur plusieurs configurations, puis le déployer.



Exercice 3 - Build et tests

Le job `build` doit :

- récupérer le dépôt et installer les dépendances
- exécuter `npm run build`
- publier `dist/` dans un artefact nommé `application`

Le job `test` doit :

- attendre la réussite de `build`
- tester Node.js 20 et 22 sur Ubuntu et Windows
- exécuter au maximum deux variantes simultanément
- ne pas interrompre les autres variantes en cas d'échec
- récupérer l'artefact puis exécuter `npm test`



Exercice 3 - Déploiement

Le job `deploy` doit :

- attendre la réussite de toute la matrice de tests
- utiliser l'Environnement choisi au déclenchement
- récupérer l'artefact `application`
- afficher la variable d'Environnement `API_URL`
- résumer la cible, l'URL et le commit dans `$GITHUB_STEP_SUMMARY`

Chaque job doit uniquement pouvoir lire le contenu du dépôt.

Dans GitHub, préparer :

```
staging    → API_URL=https://api-staging.example.com
production → API_URL=https://api.example.com
            + approbation obligatoire
```



Ce qu'il faut retenir

- `needs: build` garantit que l'artefact existe avant les tests
- `max-parallel` limite le nombre de runners utilisés simultanément
- `fail-fast: false` permet d'obtenir le résultat de chaque combinaison
- `needs: test` attend toutes les variantes de la matrice
- `vars.API_URL` dépend de l'Environment sélectionné
- la protection de `production` est configurée dans GitHub
- `contents: read` applique ici le principe du moindre privilège



Registry

Un registry est une destination de publication.

Code source



Build



Package / Image



Registry



GitHub Packages

Publier directement dans GitHub.

- lié au repository
- permissions GitHub
- packages visibles dans l'interface
- packages publics ou privés



GitHub Packages — npm

```
permissions:
  contents: read
  packages: write

steps:
  - uses: actions/checkout@v7

  - uses: actions/setup-node@v7
    with:
      node-version: 24.18.0
      registry-url: https://npm.pkg.github.com

  - run: npm ci

  - run: npm publish
    env:
      NODE_AUTH_TOKEN: ${ secrets.GITHUB_TOKEN }
```



GitHub Container Registry

```
permissions:
  contents: read
  packages: write

steps:
  - uses: actions/checkout@v7

  - uses: docker/login-action@v4
    with:
      registry: ghcr.io
      username: ${github.actor}
      password: ${secrets.GITHUB_TOKEN}

  - uses: docker/build-push-action@v7
    with:
      context: .
      push: true
      tags: ghcr.io/${github.repository}:latest
```



Registries externes

Même processus, autre destination.

Build



Authentification



Publication



Registry

- npm
- Docker Hub
- registry privé
- registry cloud



npm Registry

```
permissions:
```

```
  contents: read
```

```
  id-token: write
```

```
steps:
```

```
- uses: actions/checkout@v7
```

```
- uses: actions/setup-node@v7
```

```
  with:
```

```
    node-version: 24.18.0
```

```
    registry-url: https://registry.npmjs.org
```

```
- run: npm ci
```

```
- run: npm publish
```



Docker Hub

```
steps:
  - uses: actions/checkout@v7

  - uses: docker/login-action@v4
    with:
      username: ${vars.DOCKERHUB_USERNAME}
      password: ${secrets.DOCKERHUB_TOKEN}

  - uses: docker/build-push-action@v7
    with:
      context: .
      push: true
      tags: my-org/my-image:latest
```



Publier, ce n'est pas juste envoyer un fichier

La CI/CD doit décider :

- quoi publier
- où publier
- avec quelle version
- avec quelles permissions
- après quelles validations



Quel registry choisir ?

GitHub Packages

→ intégré au repository et aux permissions GitHub

npm / Docker Hub

→ écosystème et distribution largement utilisés

Registry privé

→ contrôle de l'infrastructure et des accès



Du tag à la production

